Quake Controller Notes

Saturday, February 28, 2026 8:36 AM

More resilient camera controlling with optional smoothing. camera_rotation(delta) should be included in process function with delta as the argument. Here I rotate the cameraHolder for pitch and the player node for yaw. You can change the nodes as needed in the camera rotation function (ie, maybe do head.transform.basis and head.rotate_object_local(...)) if you need the wallJumpCast to pitch as well

```
118 func _input(event : InputEvent) -> void:
119     #get mouse input for camera rotation
120     if event is InputEventMouseMotion:
121         rotate_y(deg_to_rad(-event.relative.x * mouse_sense))
122         head.rotate_x(deg_to_rad(-event.relative.y * mouse_sense))
123         head.rotation.x = clamp(head.rotation.x, deg_to_rad(-89), deg_to_rad(89))
```

```
# Add global variables at the top of file
var yaw_acc: float
var pitch_acc: float
var yaw_current: float
var pitch_current: float
var camera_smoothing: float = 0.003


func _input(event: InputEvent) -> void:

    # Camera
    if event is InputEventMouseMotion and Input.mouse_mode == Input.MOUSE_MODE_CAPTURED:
        yaw_acc -= event.relative.x * mouse_sense
        pitch_acc -= event.relative.y * mouse_sense
        pitch_acc = clampf(pitch_acc, deg_to_rad(-89.9), deg_to_rad(89.9))

func camera_rotation(delta) -> void:
    # Interoplate accumulators
    yaw_current = lerp(yaw_current, yaw_acc, camera_smoothing * delta)
    pitch_current = lerp(pitch_current, pitch_acc, camera_smoothing * delta)

    # reset basis
    transform.basis = Basis()
    cameraHolder.transform.basis = Basis()

    rotate_object_local(Vector3.UP, yaw_current)
    cameraHolder.rotate_object_local(Vector3.RIGHT, pitch_current)
```

The general rule of thumb is "call down, signal up". Your camera is a child of the player, so you shouldn't signal it, but call the method directly. You would use signals if for some reason you needed the camera to tell the player to do something, or if theres sibling nodes with dependencies (ie in Chainlight, I have the player and chain emit signals to each other). With camera, it's probably not needed, but with more complex nodes you can create a custom class to help with the function autofill so you know its something like startJumpAnimation() and not startJumpAnim()

```
26  # Called when the node enters the scene tree for the first time.
27  func _ready() -> void:
28      player.justJumped.connect(startJumpAnimation)
29      player.justLanded.connect(startLandAnimation)
30      player.startSlide.connect(startSlide)
31      player.endSlide.connect(endSlide)
32      player.startWallRun.connect(startWallRun)
33      player.endWallRun.connect(endWallRun)
```

```
player.endSlide.connect(endSlide)
player.startWallRun.connect(startWallRun)
player.endWallRun.connect(endWallRun)
```

```
if (Input.is_action_just_pressed("jump") or jumpQueued) and canJump:
    canJump = false
    hasJumped = true
    jumpQueued = false

    # Signal (not preferred)
    emit_signal("justJumped")

    # Call down (preferred)
    realCamera.startJumpAnimation()
```

Okay, larger structural point. The move() function seems to be your main one. You call it in both air and in ground, and it calls move_and_slide(). move_and_slide() should basically be called every physics frame, and so should the inputs of the player. So instead of splitting it up into multiple functions, you should do something like this.

_physics_process:

```
func move(delta : float, accel : float, drag : float, speed : float = speed) -> void:
    #get keyboard input
    direction = Vector3.ZERO
    var h_rot : float = global_transform.basis.get_euler().y
    var f_input : float = Input.get_axis("forward", "backward")
    var h_input : float = Input.get_action_strength("right") - Input.get_action_strength("left")
    direction = Vector3(h_input, 0, f_input).rotated(Vector3.UP, h_rot).normalized()
```

```
#get keyboard input
var input_dir: Vector2 = Input.get_vector("left", "right", "forward", "backward")
direction = (transform.basis * Vector3(input_dir.x, 0, input_dir.y)).normalized()
```

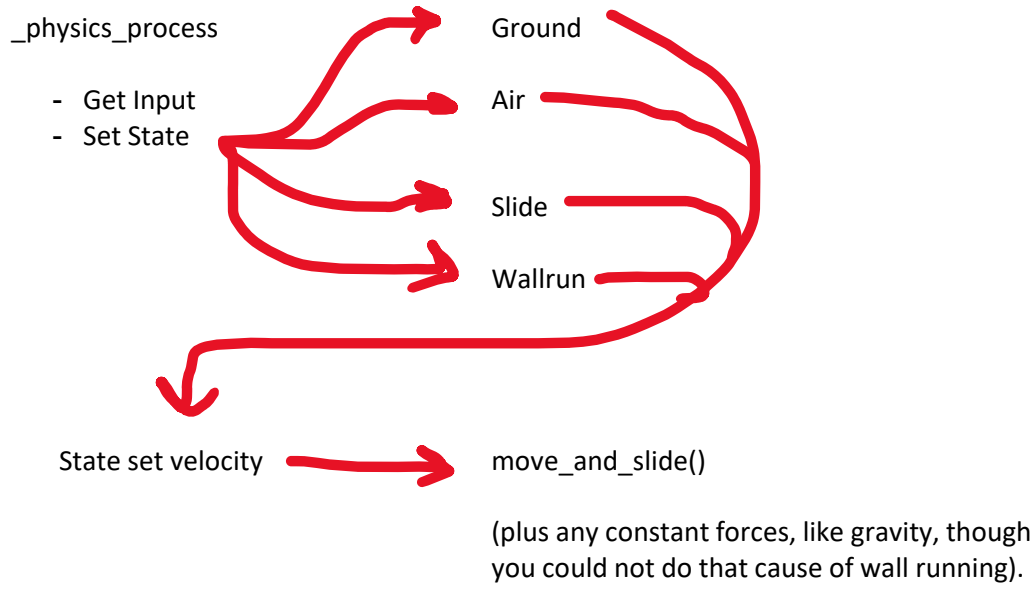
I would always keep polling the input_dir, even if you're not going to pass it. This is also just a way cleaner way of doing it. If you're trying to align one vector with another, mutliply it by the basis of the main vector rather than fucking with euler -- you get the same effect of input_dir (in the x and z with a y component of zero) aligning its x and z with the players basis.

Then determine your state:

```
match currentState:
    MOVESTATES.GROUND:
        ground(delta)
    MOVESTATES.AIR:
        air(delta)
    MOVESTATES.SLIDING:
        slide(delta)
    MOVESTATES.WALLRUNNING:
        wallrun(delta)
```

In these functions, modify the velocity of the player. Then call move_and_slide() in

_physics_process. So your flow is



This should also help clean up the state machine and also your velocities, so you KNOW only one state is setting the velocity every frame. That might be the case already, but with move() being called separately every time, it's harder to tell. It also lets you get rid of this spaghetti of checking states both in process and in move(), and you can put these processes in the state functions for better readability:

```
#Airstrafing
match currentState:
    MOVESTATES.AIR:
        var angle_diff : float = rad_to_deg(getHorizontalAngle(velocity, wish_vel))
        var samplePoint := (angle_diff - minStrafeAngle) / maxStrafeAngle
        #velocity += wish_vel.normalized() * delta * airStrafeCurve.sample(samplePoint) * airSpeed
        wish_vel *= 1.0 + (airStrafeCurve.sample(samplePoint) * airStrafeModifier)

#Crouching
if currentState == MOVESTATES.GROUND and isCrouching:
    wish_vel = direction * crouchSpeed

# Determine if decellerating or accellerating
if direction.length() > 0:
    match currentState:
        MOVESTATES.SLIDING:
        var newVelLength : float = lerp(velocity, Vector3.ZERO, drag * delta).length()
        var newVelDir : Vector3 = lerp(velocity.normalized(), wish_vel.normalized(), accel * delta)
        velocity = newVelDir.normalized() * newVelLength
        _:
        velocity = lerp(velocity, wish_vel, accel * delta)
```

That's most of the feedback without entirely restructuring things. As a bonus, here's my implementation of Quake faithful airstraffing:

```
#Airstrafing
match currentState:
    MOVESTATES.AIR:
        var wish_speed = wish_velocity.length()
        wish_velocity = wish_velocity.normalized()

        if wish_speed > speed / 10: # Can replace with any constant for air control speed
            wish_speed = speed / 10


        # The two lines that make airstraffing possible
        # Current speed (inaplty named tbh) is zero when wish_velocity
        # is 90 deg to velocity, so you get the boost of wish_speed, which
        # is capped to 30 in Quake, but speed / 10 here for unit parity

        var current_speed = velocity.dot(wish_velocity)
        var add_speed = wish_speed - current_speed

        if !(add_speed <= 0):
            printt(wish_speed, add_speed)

            var sv_accelerate = 10
            var grounded_wish_speed = wish_velocity.length()

            var accel_speed = sv_accelerate * grounded_wish_speed * delta

            if accel_speed > add_speed:
                accel_speed = add_speed

            velocity += accel_speed * wish_velocity
```